

Hausaufgaben

Vorbereitung für Abend 3 (LangChain & RAG)

Daniel Senften

Version 1.0, 2025-10-14

Inhaltsverzeichnis

1. Überblick	1
2. Aufgabe 1: AI Kurs-Assistent erweitern (□□□)	2
2.1. Lernziel	2
2.2. Aufgaben	2
3. Aufgabe 2: LangChain Grundlagen verstehen (□□)	4
3.1. Lernziel	4
3.2. 2.1 Dokumentation lesen (60 Minuten)	4
3.3. 2.2 Konzept-Map erstellen (30 Minuten)	4
3.4. 2.3 Vergleich erstellen (30 Minuten)	5
4. Aufgabe 3: RAG (Retrieval Augmented Generation) verstehen (□□□)	6
4.1. Lernziel	6
4.2. 3.1 RAG-Theorie (45 Minuten)	6
4.3. 3.2 RAG-Diagramm zeichnen (30 Minuten)	6
4.4. 3.3 RAG-Anwendungsfall entwerfen (45 Minuten)	7
5. Aufgabe 4: Vektordatenbanken erkunden (□□)	8
5.1. Lernziel	8
5.2. 4.1 Konzepte verstehen (30 Minuten)	8
5.3. 4.2 Similarity-Experiment (45 Minuten)	8
5.4. 4.3 Vektordatenbank-Vergleich (30 Minuten)	9
6. Aufgabe 5: Coding-Challenge (□□□□)	10
6.1. Lernziel	10
6.2. 5.1 Einfacher Embedding-Test (60 Minuten)	10
6.3. 5.2 Mini-RAG Prototyp (90 Minuten)	10
7. Abgabe und Bewertung	12
7.1. Abgabeformat	12
7.2. Bewertungskriterien	12
7.3. Reflexionsfragen für README.md	12
8. Hilfe und Ressourcen	14
8.1. Bei Problemen	14
8.2. Zusätzliche Ressourcen	14
9. Viel Erfolg! □	15

1. Überblick

Diese Hausaufgaben bereiten Sie optimal auf **Abend 3: LangChain & RAG** vor. Sie haben **eine Woche Zeit** und sollten etwa **3-4 Stunden** einplanen.

Ziel: Verstehen Sie die Grundlagen von LangChain und RAG, bevor wir sie gemeinsam implementieren.

2. Aufgabe 1: AI Kurs-Assistent erweitern (□□□)

2.1. Lernziel

Experimentieren Sie mit Ihrem deployed AI Kurs-Assistenten und verstehen Sie, wie AI-Anwendungen funktionieren.

2.2. Aufgaben

2.2.1. 1.1 Funktionstest (30 Minuten)

Testen Sie Ihren AI Kurs-Assistenten ausführlich:

Grundfunktionen:

- ☐ Stellen Sie 10 verschiedene Fragen zum Kurs
- ☐ Testen Sie sowohl einfache als auch komplexe Fragen
- ☐ Prüfen Sie, wie der Assistent auf unbekannte Fragen reagiert

Beispiel-Fragen:

- "Was ist LangChain?"
- "Wie funktioniert RAG?"
- "Welche Übungen machen wir in Abend 4?"
- "Erkläre mir Vektordatenbanken einfach"
- "Was ist der Unterschied zwischen GPT-4 und Claude?"
- "Wie deploye ich eine Streamlit App?"

Dokumentation: Erstellen Sie eine Datei `test-ergebnisse.md` mit:

- Ihren Fragen
- Den Antworten des Assistenten
- Ihrer Bewertung (gut/schlecht/verbesserungswürdig)

2.2.2. 1.2 Wissensbasis erweitern (45 Minuten)

Erweitern Sie die Wissensbasis in `course_knowledge.py`:

Neue Inhalte hinzufügen:

- ☐ Informationen über Ihren Lernfortschritt
- ☐ Häufige Probleme und Lösungen aus Abend 2
- ☐ Ihre persönlichen Notizen zu AI-Konzepten

- ☐ Links zu hilfreichen Ressourcen

Beispiel-Erweiterung:

```
# In course_knowledge.py hinzufügen:
PERSONAL_NOTES = """
## Meine Lernerfahrungen

### Abend 2 - Erkenntnisse:
- Python Setup war einfacher als erwartet
- Streamlit ist sehr intuitiv
- API-Integration funktioniert gut
- Deployment auf Streamlit Cloud dauert 2-3 Minuten

### Häufige Probleme:
- API-Schlüssel vergessen zu setzen
- Port 8501 manchmal nicht automatisch geöffnet
- .env Datei nicht in .gitignore

### Nützliche Links:
- Streamlit Dokumentation: https://docs.streamlit.io/
- OpenAI API Docs: https://platform.openai.com/docs
"""
```

Testen Sie die Erweiterungen:

- ☐ Deployen Sie die erweiterte Version
- ☐ Testen Sie neue Fragen basierend auf Ihren Ergänzungen

2.2.3. 1.3 App-URL teilen (15 Minuten)

Teilen Sie Ihre App:

- ☐ Senden Sie die URL an 2-3 Freunde/Familie
- ☐ Bitten Sie sie, Fragen zu stellen
- ☐ Sammeln Sie Feedback

Feedback dokumentieren: Erstellen Sie `feedback.md` mit:

- Wer hat getestet?
- Welche Fragen wurden gestellt?
- Was funktionierte gut/schlecht?
- Verbesserungsvorschläge

3. Aufgabe 2: LangChain Grundlagen verstehen (□□)

3.1. Lernziel

Verstehen Sie die Konzepte von LangChain, bevor wir sie praktisch anwenden.

3.2. 2.1 Dokumentation lesen (60 Minuten)

Pflichtlektüre:

1. **LangChain Introduction:** <https://python.langchain.com/docs/introduction/>
2. **LangChain Concepts:** <https://python.langchain.com/docs/concepts/>
3. **Chains Konzept:** <https://python.langchain.com/docs/concepts/#chains>

Leitfragen beim Lesen:

- ☐ Was ist LangChain und wofür wird es verwendet?
- ☐ Was sind "Chains" und wie funktionieren sie?
- ☐ Welche Komponenten hat LangChain?
- ☐ Wie unterscheidet sich LangChain von direkter API-Nutzung?

3.3. 2.2 Konzept-Map erstellen (30 Minuten)

Erstellen Sie eine **Konzept-Map** (Mindmap) mit folgenden Begriffen:

Zentral: LangChain

Hauptäste:

- Chains
- Prompts
- Memory
- Agents
- Tools
- Vector Stores
- Embeddings

Für jeden Begriff notieren Sie:

- Kurze Definition (1-2 Sätze)
- Anwendungsbeispiel

- Verbindung zu anderen Begriffen

Tool-Empfehlungen:

- Digital: draw.io, Miro, oder Excalidraw
- Analog: Papier und Stift (dann fotografieren)

3.4. 2.3 Vergleich erstellen (30 Minuten)

Erstellen Sie eine Tabelle: "Unser AI Kurs-Assistent vs. LangChain-Ansatz"

Aspekt	Aktueller Ansatz	Mit LangChain
API-Aufrufe	Direkte OpenAI/Anthropic Calls	Über LangChain abstrahiert
Prompt-Management	Hardcoded in Python	Template-System
Memory/Kontext	Session State	LangChain Memory
Erweiterbarkeit	Manuell programmieren	Modulare Komponenten
Komplexität	Einfach für simple Fälle	Skaliert besser

Reflexion:

- Wann würden Sie LangChain verwenden?
- Wann ist der direkte Ansatz besser?

4. Aufgabe 3: RAG (Retrieval Augmented Generation) verstehen (□□□)

4.1. Lernziel

Verstehen Sie das RAG-Konzept und seine Anwendung in AI-Systemen.

4.2. 3.1 RAG-Theorie (45 Minuten)

Video schauen: "What is Retrieval Augmented Generation (RAG)?" von IBM Technology <https://www.youtube.com/watch?v=T-D1OfcDW1M>

Zusätzliche Lektüre:

- RAG Paper (Abstract + Introduction): <https://arxiv.org/abs/2005.11401>
- LangChain RAG Tutorial: <https://python.langchain.com/docs/tutorials/rag/>

Leitfragen:

- ☐ Was ist das Problem, das RAG löst?
- ☐ Wie funktioniert RAG Schritt für Schritt?
- ☐ Was sind Embeddings und warum braucht man sie?
- ☐ Was ist eine Vektordatenbank?
- ☐ Welche Vorteile hat RAG gegenüber Fine-Tuning?

4.3. 3.2 RAG-Diagramm zeichnen (30 Minuten)

Zeichnen Sie ein **Flussdiagramm** des RAG-Prozesses:

Komponenten einbeziehen:

1. Dokumente/Wissensbasis
2. Embedding-Modell
3. Vektordatenbank
4. Benutzer-Anfrage
5. Similarity Search
6. Kontext-Retrieval
7. LLM mit Prompt + Kontext
8. Generierte Antwort

Beschriften Sie jeden Schritt mit 1-2 Sätzen Erklärung.

4.4. 3.3 RAG-Anwendungsfall entwerfen (45 Minuten)

Entwerfen Sie einen RAG-Anwendungsfall für Ihren Beruf/Studium:

Beispiele:

- Firmen-Wissensdatenbank für Kundenservice
- Rechtsdokumente-Assistent für Anwaltskanzlei
- Medizinische Leitlinien-Assistent für Ärzte
- Produktdokumentation-Assistent für Entwickler

Ihr Konzept sollte enthalten:

1. **Problemstellung** (2-3 Sätze)
2. **Zielgruppe** (wer würde es nutzen?)
3. **Datenquellen** (welche Dokumente/Informationen?)
4. **Beispiel-Fragen** (5 typische Benutzer-Anfragen)
5. **Erwartete Antworten** (wie sollte das System antworten?)
6. **Herausforderungen** (was könnte schwierig werden?)

Format: Erstellen Sie eine Präsentation (3-5 Folien) oder ein Dokument

5. Aufgabe 4: Vektordatenbanken erkunden (□□)

5.1. Lernziel

Verstehen Sie, wie Vektordatenbanken funktionieren und warum sie für AI wichtig sind.

5.2. 4.1 Konzepte verstehen (30 Minuten)

Lesen Sie:

- "What are Vector Databases?" - Pinecone Blog
- Unser Kurs-Material: [docs/chapters/04-vector-stores.adoc](#)

Verstehen Sie:

- ☐ Was sind Vektoren in diesem Kontext?
- ☐ Wie werden Texte zu Vektoren?
- ☐ Was bedeutet "semantische Ähnlichkeit"?
- ☐ Welche Vektordatenbanken gibt es?

5.3. 4.2 Similarity-Experiment (45 Minuten)

Führen Sie ein einfaches Experiment durch:

- 1. Wählen Sie 10 Sätze** zu verschiedenen Themen:
 - 3 über AI/ML
 - 3 über Kochen
 - 3 über Sport
 - 1 gemischter Satz
- 2. Verwenden Sie ein Online-Tool** für Embeddings:
 - Hugging Face Spaces: "sentence-transformers"
 - Oder: OpenAI Playground (falls API verfügbar)
- 3. Vergleichen Sie die Ähnlichkeiten:**
 - Welche Sätze sind sich am ähnlichsten?
 - Entspricht das Ihren Erwartungen?
 - Was überrascht Sie?

Dokumentieren Sie Ihre Erkenntnisse in [similarity-experiment.md](#)

5.4. 4.3 Vektordatenbank-Vergleich (30 Minuten)

Erstellen Sie eine **Vergleichstabelle** der wichtigsten Vektordatenbanken:

Datenbank	Typ	Vorteile	Nachteile	Anwendung
FAISS	Bibliothek	Schnell, kostenlos	Nur in-memory	Prototyping
Chroma DB	Embedded	Einfach, persistent	Begrenzte Skalierung	Kleine Projekte
Pinecone	Cloud-Service	Skalierbar, managed	Kostenpflichtig	Produktion
Weaviate	Open Source	Flexibel, GraphQL	Komplex zu setup	Enterprise
Qdrant	Open Source	Performant, Rust	Weniger bekannt	High-Performance

6. Aufgabe 5: Coding-Challenge (□□□□)

6.1. Lernziel

Wenden Sie Ihr Wissen praktisch an und bereiten Sie sich auf die Implementierung vor.

6.2. 5.1 Einfacher Embedding-Test (60 Minuten)

Erstellen Sie ein Python-Skript `embedding_test.py`:

```
# Ziel: Verstehen Sie, wie Embeddings funktionieren

import openai # oder anthropic
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

# 1. Erstellen Sie Embeddings für verschiedene Texte
texts = [
    "Python ist eine Programmiersprache",
    "JavaScript wird für Webentwicklung verwendet",
    "Hunde sind treue Haustiere",
    "Katzen sind unabhängige Tiere",
    "Machine Learning ist ein Teilbereich der KI"
]

# 2. Berechnen Sie Ähnlichkeiten
# 3. Finden Sie die ähnlichsten Textpaare
# 4. Visualisieren Sie die Ergebnisse
```

Erwartetes Ergebnis:

- Programmiersprachen-Texte sind sich ähnlich
- Haustier-Texte sind sich ähnlich
- AI-Text steht für sich

6.3. 5.2 Mini-RAG Prototyp (90 Minuten)

Erstellen Sie einen einfachen RAG-Prototyp `mini_rag.py`:

Funktionalität:

1. **Dokumente einlesen** (3-5 Textdateien über verschiedene Themen)
2. **Embeddings erstellen** für jedes Dokument
3. **Benutzer-Frage** als Embedding
4. **Ähnlichste Dokumente finden**

5. **Kontext + Frage** an LLM senden

6. **Antwort** ausgeben

Vereinfachungen erlaubt:

- Nur lokale Dateien (keine Datenbank)
- Einfache Textverarbeitung
- Basis-Implementierung ohne Optimierung

Test-Fragen vorbereiten:

- Fragen, die in Ihren Dokumenten beantwortet werden können
- Fragen, die nicht beantwortet werden können

7. Abgabe und Bewertung

7.1. Abgabeformat

Erstellen Sie einen Ordner **hausaufgaben-woche-2-3/** mit:

```
hausaufgaben-woche-2-3/
├── README.md                # Übersicht und Reflexion
├── aufgabe-1/
│   ├── test-ergebnisse.md
│   ├── feedback.md
│   └── course_knowledge.py  # Erweiterte Version
├── aufgabe-2/
│   ├── konzept-map.png     # Oder PDF
│   └── langchain-vergleich.md
├── aufgabe-3/
│   ├── rag-diagramm.png    # Oder PDF
│   ├── rag-anwendungsfall.pdf # Oder PPTX
│   └── rag-reflexion.md
├── aufgabe-4/
│   ├── similarity-experiment.md
│   └── vektordatenbank-vergleich.md
└── aufgabe-5/
    ├── embedding_test.py
    ├── mini_rag.py
    └── test-dokumente/
```

7.2. Bewertungskriterien

Kriterium	Beschreibung	Punkte
Vollständigkeit	Alle Aufgaben bearbeitet	25%
Verständnis	Konzepte richtig verstanden und erklärt	30%
Praktische Umsetzung	Code funktioniert und ist dokumentiert	25%
Reflexion	Eigene Gedanken und Erkenntnisse	20%

7.3. Reflexionsfragen für README.md

Beantworten Sie in Ihrem **README.md**:

1. Was war die grösste Erkenntnis dieser Woche?
2. Welche Aufgabe war am schwierigsten und warum?
3. Wie hat sich Ihr Verständnis von AI verändert?

4. **Welche Fragen haben Sie** für Abend 3?
5. **Wie würden Sie RAG** in Ihrem Beruf/Studium einsetzen?

8. Hilfe und Ressourcen

8.1. Bei Problemen

1. Technische Probleme:

- Nutzen Sie die Backup-Umgebungen (Codespaces/Gitpod)
- Fragen Sie in der Kurs-Gruppe

2. Verständnisprobleme:

- Nutzen Sie Ihren AI Kurs-Assistenten
- Schauen Sie zusätzliche YouTube-Videos
- Lesen Sie die verlinkten Artikel nochmals

3. Zeitprobleme:

- Priorisieren Sie: Aufgaben 1-3 sind wichtiger als 4-5
- Qualität vor Quantität

8.2. Zusätzliche Ressourcen

Videos:

- "LangChain Explained in 13 Minutes" - TechWithTim
- "Vector Databases Explained" - Fireship

Dokumentation:

- LangChain: <https://python.langchain.com/>
- Streamlit: <https://docs.streamlit.io/>
- OpenAI: <https://platform.openai.com/docs>

Tools:

- Online Embeddings: <https://huggingface.co/spaces>
- Diagramme: <https://draw.io>, <https://excalidraw.com>

9. Viel Erfolg! ☐

Diese Hausaufgaben bereiten Sie optimal auf Abend 3 vor. Nehmen Sie sich die Zeit, die Konzepte wirklich zu verstehen - es wird sich auszahlen!

Bei Fragen: Nutzen Sie Ihren AI Kurs-Assistenten oder fragen Sie in der Kurs-Gruppe.

Deadline: Bis zum Beginn von Abend 3 (eine Woche Zeit)